



A
Report
On
Emotion Detection Web Application Project

In fulfillment of the requirements

**For Coursework in
Django and FastAPI**

Submitted By

Saurav GC

Student ID: 25123840

Section: I2

Submitted To

Umesh Regmi

Module Coordinator

2 Oct, 2026

Table of Contents

1. Introduction	3
1.1 Background of the Project	3
1.2 Problem Statement	3
1.3 Purpose of the Project	3
2. Objectives	4
3. Methodology	4
3.1 Dataset	4
3.2 Emotion Mapping	5
3.3 Text Preprocessing	5
3.4 Feature Extraction	6
3.5 Model Selection	6
3.6 Evaluation metrics	7
3.7 Tools and Technologies	7
4. Project Implementation	8
4.1 System Architecture	8
4.2 Machine Learning Pipeline	8
4.3 Django Web Application	9
4.4 Prediction Flow	9
4.5 Authentication System	10
4.6 Administration, Demo Data, and Testing	10
4.7 Screenshots	11
5. Results/Output	15
5.1 Dataset Statistics After Preprocessing	15
5.2 Class Distribution	16
5.3 Model Performance	17
5.4 Crisis Safety Backoff	19
5.5 Automated Tests	20
5.6 Key Observations	20
6. Challenges and Limitations	20
6.1 Challenges	20
6.2 Limitations	21
7. Conclusion	21
7.1 Summary	21
7.2 Major achievements and learning outcomes	22
8. Future Scope	22
References	23
Appendices	23

1. Introduction

1.1 Background of the Project

Emotion detection is a subfield of Natural Language Processing (NLP) that identifies and classifies the human emotions expressed in text. Unlike simple sentiment analysis, which categorizes text as merely positive or negative, emotion detection provides a more granular understanding by distinguishing specific states such as joy, anger, fear, sadness, surprise, disgust, and neutral.

With the rapid growth of social media, customer feedback platforms, and digital communication, the volume of emotion-laden text has grown exponentially. Automated emotion detection has become valuable across mental health monitoring, customer-service analytics, content moderation, and marketing research. This project brings that capability into a teachable full-stack setting: a machine-learning model trained on real annotated data, served through a web application that students can run end to end.

1.2 Problem Statement

Existing sentiment analysis tools are limited to binary or ternary classifications of positive, negative, or neutral, which fail to capture the nuanced emotional content of human language. A person writing “I cannot believe this happened” may be expressing joy, anger, or surprise; a simple sentiment classifier cannot distinguish between these.

There is a clear need for an accessible, web-based tool that classifies text into specific emotion categories with confidence scores for every class, enabling users to understand the emotional tone of their writing objectively, review their past submissions, and see how certain the model actually is.

1.3 Purpose of the Project

The purpose of this project is to develop a full-stack Django web application that accepts free-form text input from authenticated users and returns a predicted emotion alongside confidence scores for all seven emotion classes. The system is powered by a machine-learning pipeline trained on the GoEmotions dataset, a large-scale collection of Reddit comments annotated with 28 emotion labels, published by Google Research (Demszky et al., 2020), whose labels are collapsed into seven core emotions. Beyond accuracy, the project also implements a safety mechanism that guarantees self-harm phrasing is never answered with a positive or neutral label.

2. Objectives

The main objectives of this project are :

- To train a multi-class text classification model capable of identifying seven core emotions joy, anger, fear, sadness, surprise, disgust, and neutral from

free-form text input, returning calibrated confidence scores for every class.

- To implement a robust NLP preprocessing pipeline covering noise removal (lower-casing, URL stripping, removal of GoEmotions placeholders and punctuation) and TF-IDF feature extraction, shared verbatim between training and serving so the two can never drift apart.
- To develop a Django-based web application with a complete user authentication system covering registration, login, and logout.
- To integrate the trained model into the web application so that authenticated users can submit text and receive real-time emotion predictions with per-class score bars.
- To persist each user’s prediction history in a database, so users can review their own past predictions and administrators can inspect every run through the Django admin.
- To achieve a macro-average F1 score of at least 0.60 on the held-out test set, far above the most frequent baseline of 0.077 for this 7-class problem.

3. Methodology

3.1 Dataset

The project uses the GoEmotions dataset published by Google Research (Demszky et al., 2020). It contains **54,263** Reddit comments manually annotated with 28 emotion labels by trained human raters, pre-split by the authors into training, validation (dev), and test sets. For this project the 28 labels are collapsed into seven core emotions, and multi-label rows (16.4% of training data) are dropped so each row carries exactly one target class. Table 1 summarizes the splits.

Split	Total rows	Multi-label	Single-label	After Cleaning
Train	43,410	7,102 (16.36%)	36,308	36,284
Dev	5,426	878 (16.18%)	4,548	4,547
Test	5,427	837 (15.42%)	4,590	4,588
Total	54,263	8,817	45,446	45,419

Table 1: GoEmotions splits before and after preprocessing

3.2 Emotion Mapping

The 28 original GoEmotions labels are mapped to 7 core emotions as follows:

Core Emotion	GoEmotions Labels Mapped
Joy	admiration, amusement, approval, caring, desire, excitement, gratitude, love, optimism, pride, relief, joy
Anger	anger, annoyance, disapproval
Fear	fear, nervousness
Sadness	disappointment, embarrassment, grief, remorse, sadness
Surprise	confusion, curiosity, realization, surprise
Disgust	disgust
Neutral	neutral

Table 2: 28-to-7 emotion collapse mapping.

3.3 Text Preprocessing

The text preprocessing pipeline includes the following steps:

- Converting all text to lowercase
- Removing URLs and hyperlinks using regular expressions
- Removing GoEmotions-specific placeholders such as [NAME]
- Stripping punctuation and non-alphabetic characters
- Dropping multi-label rows where a comment was annotated with more than one emotion

3.4 Feature Extraction

Text features are extracted with TF-IDF (Term Frequency–Inverse Document Frequency; Salton and Buckley, 1988) using a FeatureUnion of two vectorisers:

- **Word-level TF-IDF** up to 30,000 features, n-grams (1, 3), min_df=2, sublinear TF scaling;

- **Character-level TF-IDF** (char_wb analyser) up to 80,000 features, n-grams (2, 6), same smoothing.

The union yields **110,000 features**. The vectorizer is fitted on the training split only while hyperparameters are being chosen (no leakage into dev), and the final winner is re-fitted on train + dev (40,831 rows) exactly once before the test set is scored.

3.5 Model Selection

All model choices were made on the dev set; the test set was touched only once, at the end. The procedure, implemented in ml/train.py and mirrored step by step in the project notebook ml/pipeline.ipynb, was:

1. **Baseline:** a DummyClassifier predicting the most frequent class establishes the performance floor (dev macro-F1 0.0767).
2. **Feature configuration sweep:** eight configurations comparing word-only vs. word+char, stopwords removal vs. none, and several n-gram widths and vocabulary sizes; the wide word(1,3)+char(2,6) configuration without stopwords removal won on dev.
3. **Regularization:** C was swept over {0.03, 0.1, 0.3, 1, 3}; C = 0.1 gave the best dev macro-F1 (0.5612).
4. **Class imbalance:** balanced class weights, then an extra multiplier for the weak classes (anger, surprise, disgust); ×1.5 was selected.
5. **Calibration:** LinearSVC exposes no probabilities, so CalibratedClassifierCV (3-fold) was fitted with sigmoid and isotonic methods; sigmoid won (dev 0.5686). Platt-scaled probabilities power the confidence bars in the UI.
6. **Decision boosts:** a coordinate search tuned one multiplicative boost per class on dev, lifting the score to 0.5915. The boosted probabilities are wrapped in a BoostedClassifier so argmax, confidence bars and saved history stay consistent.
7. **Soft ensemble check:** an ensemble of calibrated members at nearby C values reached 0.5908; a margin rule (the ensemble must beat the single model by more than 0.0005) kept the simpler single model.
8. **Refit and serialization:** the winner was re-fit on train + dev and pickled as model.pkl + vectorizer.pkl.
9. **Crisis backoff:** a rule-based safety layer (ml/crisis.py) forces sadness above the model's argmax when the input matches a curated self-harm phrase; untouched inputs pass through bit for bit.

3.6 Evaluation metrics

Given the strong class imbalance (joy and neutral together hold over 70% of training rows), accuracy alone is unsuitable. The primary metric is the macro-average F1 score,

which weights every class equally. Per class precision, recall, and F1, the confusion matrix and accuracy are reported alongside. Two additional checks guard the deployment path: a hand-labeled crisis slice (38 rows) that must be served correctly and an automated Django test suite of 32 tests covering authentication, prediction, history, and the serving path.

3.7 Tools and Technologies

Category	Tool / Technology	Purpose
Language	Python 3.14	ML pipeline and web backend
Web framework	Django 6.0	Apps, auth, ORM, admin, templates
Machine learning	scikit-learn 1.9	TF-IDF, LinearSVC, calibration, metrics
Data handling	pandas 3.0 / NumPy 2.x	TSV loading, splits, arrays
NLP utilities	NLTK 3.10	Optional stopword list (disabled in production)
Visualization	matplotlib 3.10 / seaborn 0.13	EDA charts, confusion matrix
Database	SQLite 3 (db.sqlite3)	Users and prediction history
Serialization	Python pickle	Model and vectorizer artifacts
Experimentation	Jupyter notebook	Self-contained EDA → training → evaluation report
Testing	Django test framework	32 automated tests
Frontend	HTML5 / custom CSS	Editorial “newsprint” design system, no JS framework
Version control	Git	Commit history of the development campaign

4. Project Implementation

4.1 System Architecture

The application follows a three-layer architecture (Figure 1). a data and model layer, a web application layer, and a persistence layer. The machine learning model is trained offline and serialized as a pickle file. The Django web application loads this pickle at runtime and uses it to serve predictions to authenticated users.

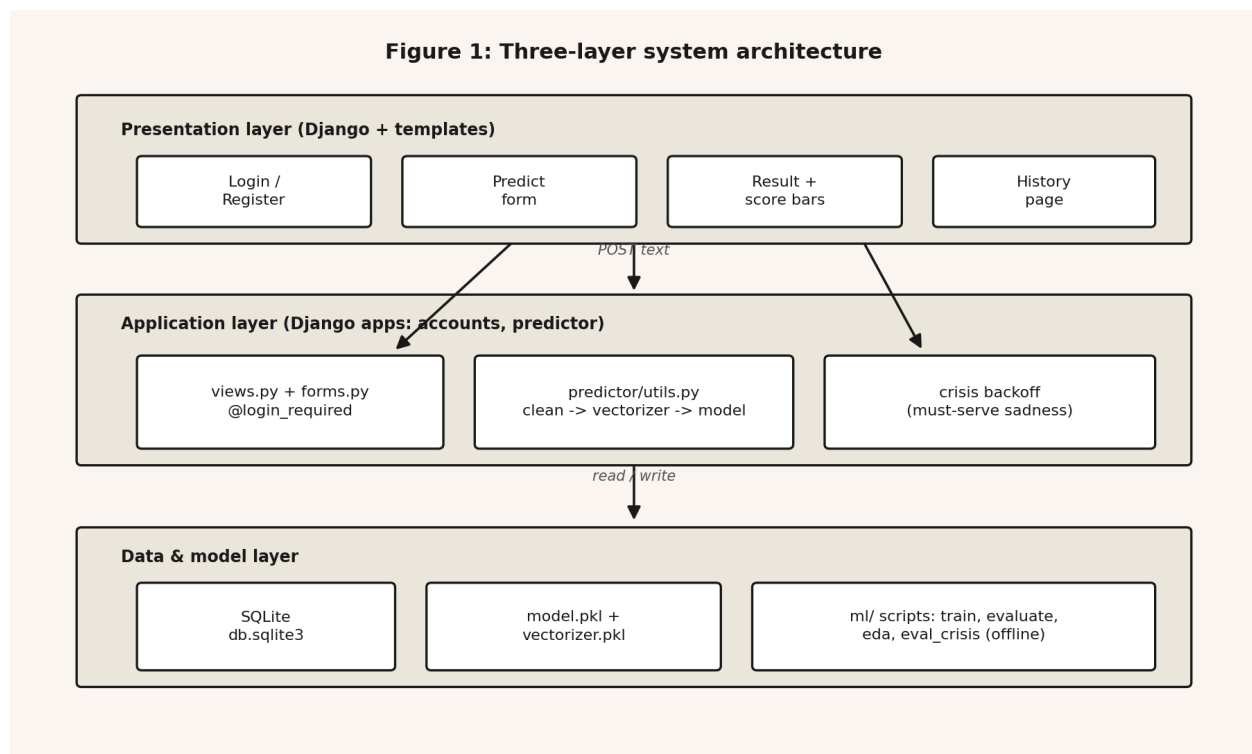


Figure 1: Three-layer system architecture—presentation, application, and data/model layers.

4.2 Machine Learning Pipeline

The pipeline (Figure 2) was developed in Python with scikit-learn, first explored in a self-contained Jupyter notebook (ml/pipeline.ipynb: EDA → preprocessing → training → evaluation → crisis backoff) and then productionized as small command-line scripts (eda.py, preprocess.py, train.py, evaluate.py, eval_crisis.py, crisis.py) that share the same cleaning code. A baseline DummyClassifier was evaluated first to establish the floor, followed by the sweep described. Every hyperparameter was chosen on dev; the final refit on train + dev and the single test evaluation happen only at the end of the run.

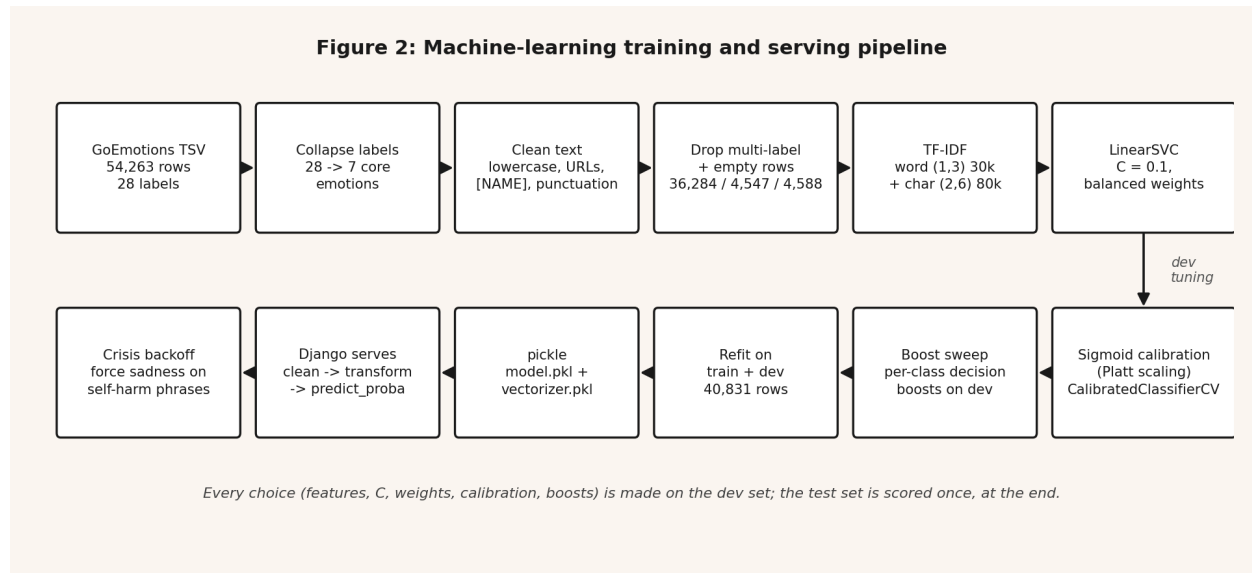


Figure 2: Machine-learning training and serving pipeline, from raw TSV files to the crisis backoff applied at request time.

4.3 Django Web Application

The project is a Django project named config with two applications:

- **accounts:** registration, login, and logout views built on Django's built-in AuthenticationForm/UserCreationForm, with safe handling of ?next= redirect targets;
- **predictor:** the prediction form (EmotionForm, 3–2000 characters), the view that runs inference and persists each run, the per-user history page with emotion summary chips, the model-loading utilities (utils.py) and a management command seed_db that creates demo accounts and backdated sample predictions.

Shared templates live in templates/base.html and static assets (a custom CSS design system and SVG favicon) in static/. The Django admin registers the prediction model so every run can be inspected and filtered by emotion.

4.4 Prediction Flow

When a logged-in user submits text, the following happens:

1. The form validates the input (3–2000 characters after stripping).
2. predict_emotion() cleans the text with the exact function used in training (lower case, URL/[NAME]/punctuation removal).

3. The fitted TF-IDF vectorizer transforms it into a 110,000-dimensional sparse vector.
4. The pickled BoostedClassifier returns calibrated, boost-adjusted, and renormalized probabilities for all seven classes; the highest one is the verdict, and its probability is the displayed confidence.
5. The crisis backoff runs on the raw input: if a curated phrase matches (e.g., "suicid," "kill myself," "want to die"), sadness is placed above the current maximum (+0.10 margin) and the distribution renormalized; otherwise, the probabilities are passed through unchanged.
6. The row (user, text, emotion, confidence, full score list, timestamp) is saved to SQLite and the result page renders the verdict with per-class score bars.

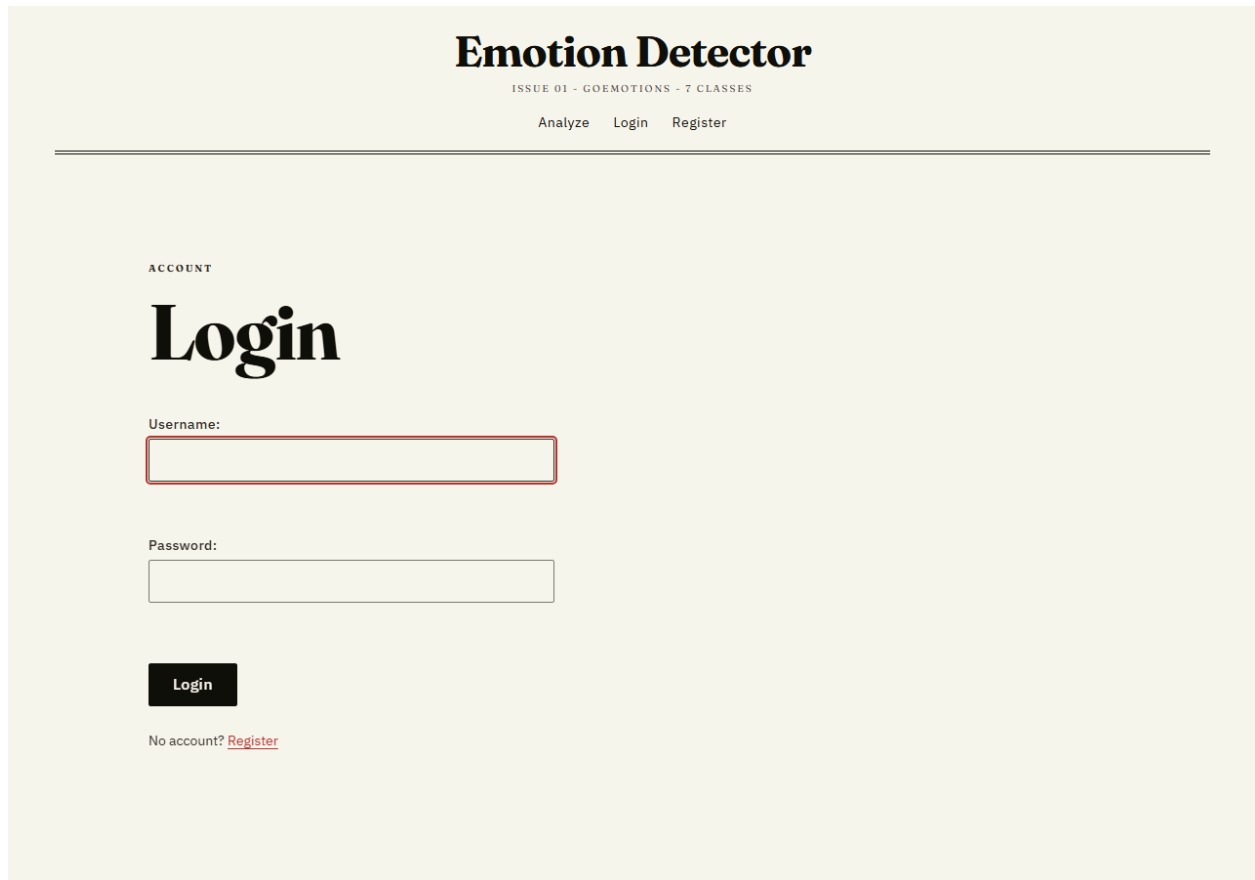
4.5 Authentication System

All prediction routes are protected with Django's `@login_required` decorator; anonymous visitors are redirected to the login page. New users register with a username and password through Django's standard user-creation flow and are logged in immediately. Session state is handled by Django's built-in session framework with CSRF protection on every POST and `never_cache` on the auth views.

4.6 Administration, Demo Data and Testing

- **Admin** a superuser can browse, search, and filter every prediction.
- **Seed command** `manage.py seed_db` creates three demo accounts (admin, maya, ravi) and 30+ backdated sample predictions so the history and admin screens are populated out of the box; the shipped `db.sqlite3` already contains them.
- **Tests** `manage.py test` runs 32 tests across both apps, including a test that pushes the whole hand-labeled crisis slice through the real serving path.
- **Crisis evaluation** `python ml/eval_crisis.py` prints a per-row before/after report and exits non-zero if any check fails.

4.7 Screenshots



The screenshot shows the login page of the 'Emotion Detector' application. The page has a light beige background. At the top center, the title 'Emotion Detector' is displayed in a large, bold, black serif font. Below the title, in a smaller, all-caps sans-serif font, is the text 'ISSUE 01 - GOEMOTIONS - 7 CLASSES'. A horizontal line separates the header from the main content area. Below this line, centered, are three links: 'Analyze', 'Login', and 'Register'. The 'Login' link is highlighted with a dark background. In the main content area, the word 'ACCOUNT' is written in a small, all-caps sans-serif font. Below it, the word 'Login' is displayed in a large, bold, black serif font. Underneath the title, there are two input fields: one for 'Username:' and one for 'Password:'. Both fields are outlined with a thin red border. Below the password field is a black button with the word 'Login' in white. At the bottom of the form, there is a link that says 'No account? [Register](#)'.

Emotion Detector

ISSUE 01 - GOEMOTIONS - 7 CLASSES

Analyze Login Register

ACCOUNT

Login

Username:

Password:

Login

No account? [Register](#)

Figure 3: Login page all prediction features require an account

Emotion Detector

ISSUE 01 - GOEMOTIONS - 7 CLASSES

Analyze Login Register

ACCOUNT

Register

Username:

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Figure 4: Registration page (Django's UserCreationForm, CSRF-protected).

Emotion Detector

ISSUE 01 - GOEMOTIONS - 7 CLASSES

Analyze History maya Logout

01 / INPUT

Your words, read closely.

Paste anything you wrote or received: a journal entry, a review, a message. The model picks the dominant emotion out of seven and scores every class.

Your text

Type or paste anything: a journal entry, a review, a message...

MODEL

TF-IDF over cleaned text, scored by LinearSVC. Seven classes: joy, anger, fear, sadness, surprise, disgust, neutral.

Figure 5: Prediction form with the model card shown alongside the textarea

Emotion Detector

ISSUE 01 - GOEMOTIONS - 7 CLASSES

[Analyze](#) [History](#) [maya](#) [Logout](#)

01 / INPUT

Your words, read closely.

Paste anything you wrote or received: a journal entry, a review, a message. The model picks the dominant emotion out of seven and scores every class.

Your text

I cannot believe I actually got the job offer, I am thrilled and my hands are shaking!

Analyze

MODEL

TF-IDF over cleaned text, scored by LinearSVC. Seven classes: joy, anger, fear, sadness, surprise, disgust, neutral.

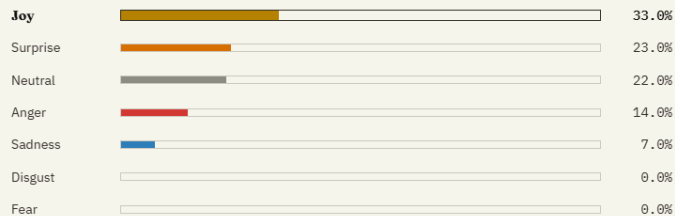
02 / VERDICT

Predicted emotion

■ Joy

33.5%

CONFIDENCE



Analyze another

View history

Figure 6: Prediction result—verdict, confidence, and per-class score bars rendered from calibrated probabilities

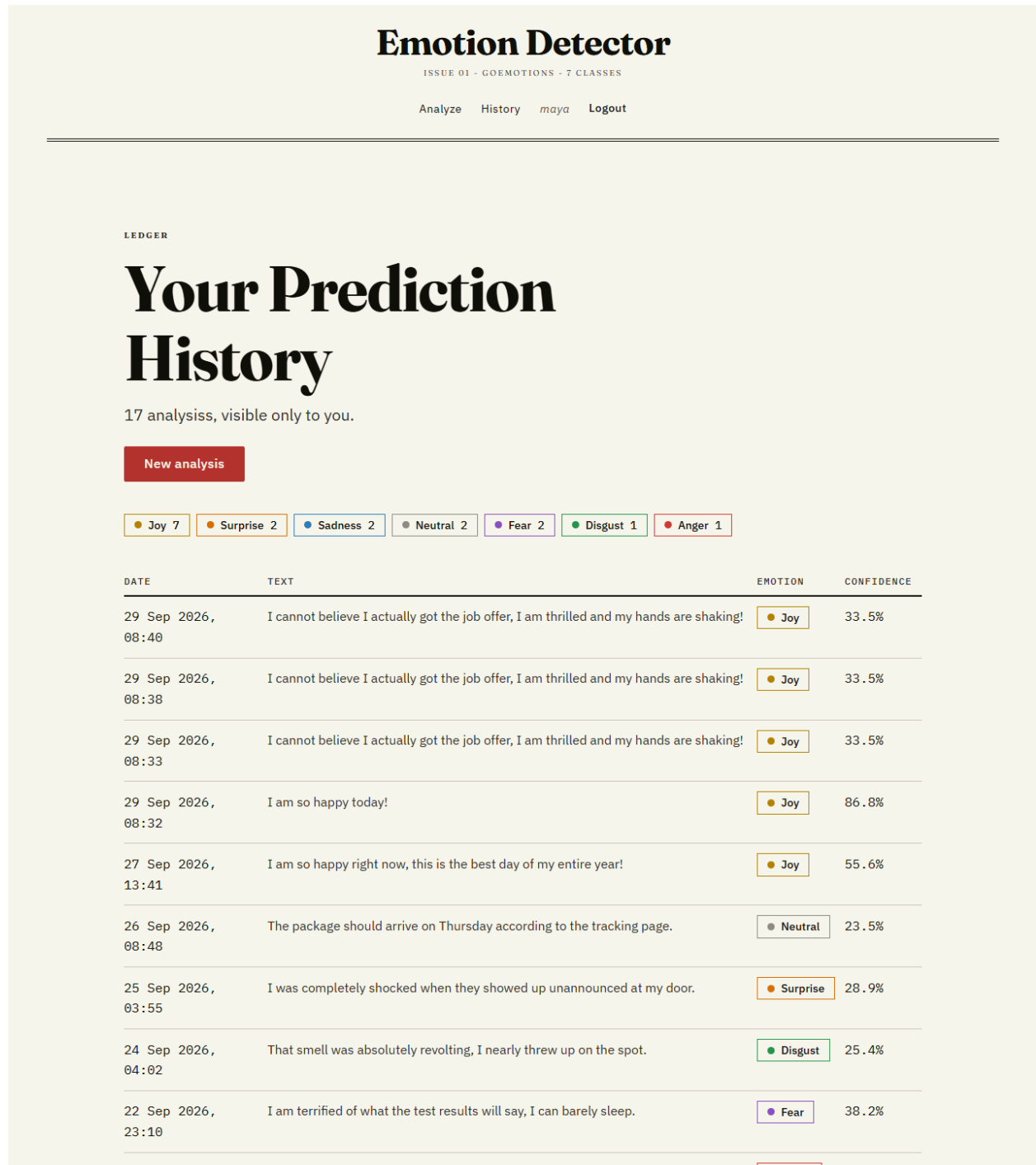


Figure 5: Per-user prediction history with emotion summary chips; each row keeps the original text, verdict, confidence, and timestamp

Django administration

WELCOME, ADMIN. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

[Home](#) > [Predictor](#) > Predictions

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

[Groups](#) + Add
 [Users](#) + Add

PREDICTOR

[Predictions](#) + Add

Select prediction to change

Action: ----- 0 of 41 selected

☐	USER	PREDICTED EMOTION	CONFIDENCE	CREATED AT	SHORT TEXT
☐	maya	Joy	0.33484160953353553	Sept. 29, 2026, 8:40 a.m.	I cannot believe I actually got the job offer, I a
☐	maya	Joy	0.33484160953353553	Sept. 29, 2026, 8:38 a.m.	I cannot believe I actually got the job offer, I a
☐	maya	Joy	0.33484160953353553	Sept. 29, 2026, 8:33 a.m.	I cannot believe I actually got the job offer, I a
☐	maya	Joy	0.867544041700839	Sept. 29, 2026, 8:32 a.m.	I am so happy today!
☐	admin	Neutral	0.6043762493181843	Sept. 28, 2026, 3:33 p.m.	suicide
☐	admin	Joy	0.4202450476771964	Sept. 28, 2026, 3:30 p.m.	i will do suicide
☐	admin	Neutral	0.5540594576299369	Sept. 28, 2026, 3:30 p.m.	suicide
☐	admin	Joy	0.3004450421192071	Sept. 28, 2026, 3:29 p.m.	i want to die
☐	admin	Anger	0.64560173510427	Sept. 28, 2026, 3:28 p.m.	cant say me nothing
☐	admin	Neutral	0.4301554336838207	Sept. 28, 2026, 3:25 p.m.	life is not worthy living
☐	admin	Neutral	0.4301554336838207	Sept. 28, 2026, 3:25 p.m.	life is not worthy living
☐	admin	Anger	0.3843865170639134	Sept. 28, 2026, 3:24 p.m.	what a hell of life it is
☐	admin	Neutral	0.8077574505205656	Sept. 28, 2026, 3:23 p.m.	hello
☐	admin	Neutral	0.8077574505205656	Sept. 28, 2026, 2:55 p.m.	hello
☐	admin	Neutral	0.21085442576209024	Sept. 27, 2026, 3:48 p.m.	hater
☐	admin	Neutral	0.38482981506205644	Sept. 27, 2026, 3:48 p.m.	hello

ADD PREDICTION +

FILTER

[Show counts](#)

By predicted emotion

[All](#)
[Joy](#)
[Anger](#)
[Fear](#)
[Sadness](#)
[Surprise](#)
[Disgust](#)
[Neutral](#)

By created at

[Any date](#)
[Today](#)
[Past 7 days](#)
[This month](#)
[This year](#)

Figure 8: Django admin every prediction, filterable by predicted emotion and creation date

5. Results/Output

5.1 Dataset Statistics After Preprocessing

After dropping multi-label rows and rows left empty by cleaning, the usable dataset is **36,284 / 4,547 / 4,588** rows for train/dev/test (Table 1). Data-quality checks found no null cells, no empty texts and no duplicate IDs; duplicate texts are rare (183 train, 3 dev, 6 test). Comments are short: a mean of 67.3 characters and 12.6 words (median 12), so the model must decide from very little context.

5.2 Class Distribution

The training set is heavily imbalanced, with joy (35.6%) and neutral (35.3%) dominating, while fear (1.4%) and disgust (1.4%) are rare. Balanced class weights plus a $\times 1.5$ multiplier on the weak classes are applied during training.

Emotion	Training rows	Share	Test F1
joy	12,918	35.6%	0.7623
neutral	12,803	35.3%	0.6589
anger	3,877	10.7%	0.4809
surprise	3,553	9.8%	0.4905
sadness	2,121	5.8%	0.5734
fear	515	1.4%	0.6887
disgust	497	1.4%	0.4833

Table 3: Class distribution of the cleaned training set with per-class test F1.

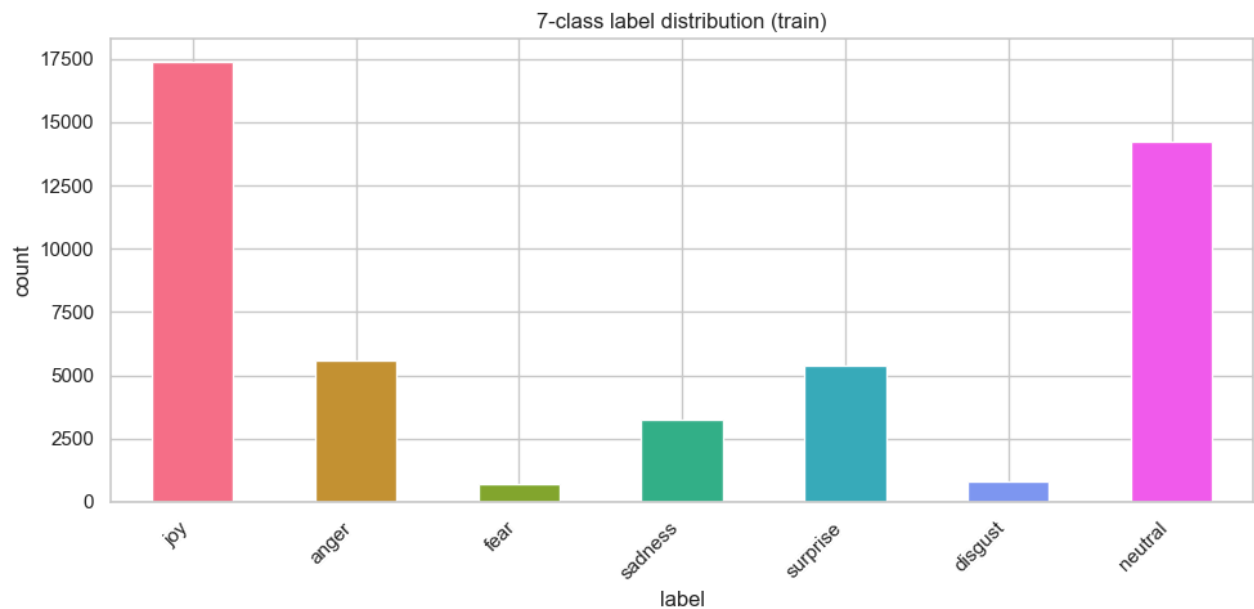


Figure 9: Original GoEmotions label distribution before collapsing 28 labels into seven core emotions.

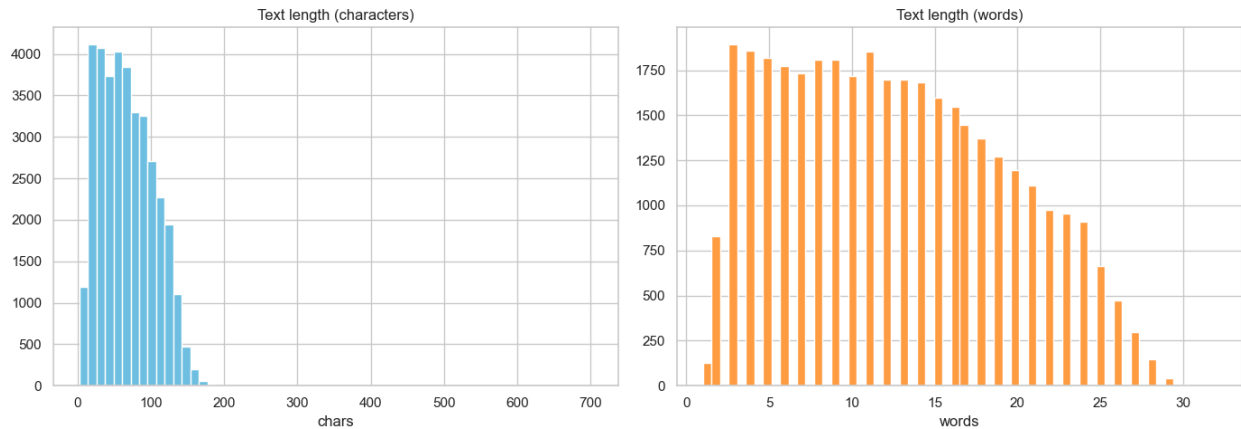


Figure 10: Text-length distribution short inputs are the norm, with a thin tail up to 703 characters.

5.3 Model Performance

Table 4 summarizes the dev-set campaign (every decision made on dev), and Table 5 gives the single, final test-set evaluation. The served model reaches a test macro-F1 of 0.5911 with 65.28% accuracy against a most-frequent baseline of 0.0767, an improvement of roughly 7.7 $\times$. The 0.60 objective was missed by 0.009.

Dev set step	Macro F1
Most frequent baseline	0.0767
TF-IDF + LinearSVC, best C (0.1)	0.5612
+ weak-class weights $\times 1.5$, sigmoid calibration	0.5686
+ per-class decision boosts (single model, selected)	0.5915
+ soft ensemble of calibrated members (not selected)	0.5908
Final refit on train+dev, dev score (in-sample)	0.7995

Table 4: Development-set results at each tuning step.

Final test-set metric	Raw argmax	Served (boosts)
Macro-F1	0.5651	0.5911
Accuracy	0.6528	0.6528
Weighted F1	0.6375	0.6511

Table 5: Single final evaluation on the held-out test set (4,588 rows), before and after the decision boosts that the web application serves.

Emotion	Precision	Recall	F1	Support
joy	0.7738	0.7511	0.7623	1,603
anger	0.5042	0.4596	0.4809	520
fear	0.7027	0.6753	0.6887	77
sadness	0.6902	0.4903	0.5734	259
surprise	0.4911	0.4900	0.4905	449
disgust	0.6591	0.3816	0.4833	76
neutral	0.6217	0.7007	0.6589	1,604
Macro avg	0.6347	0.5641	0.5911	4,588

Table 6: Per-class test results of the served model.

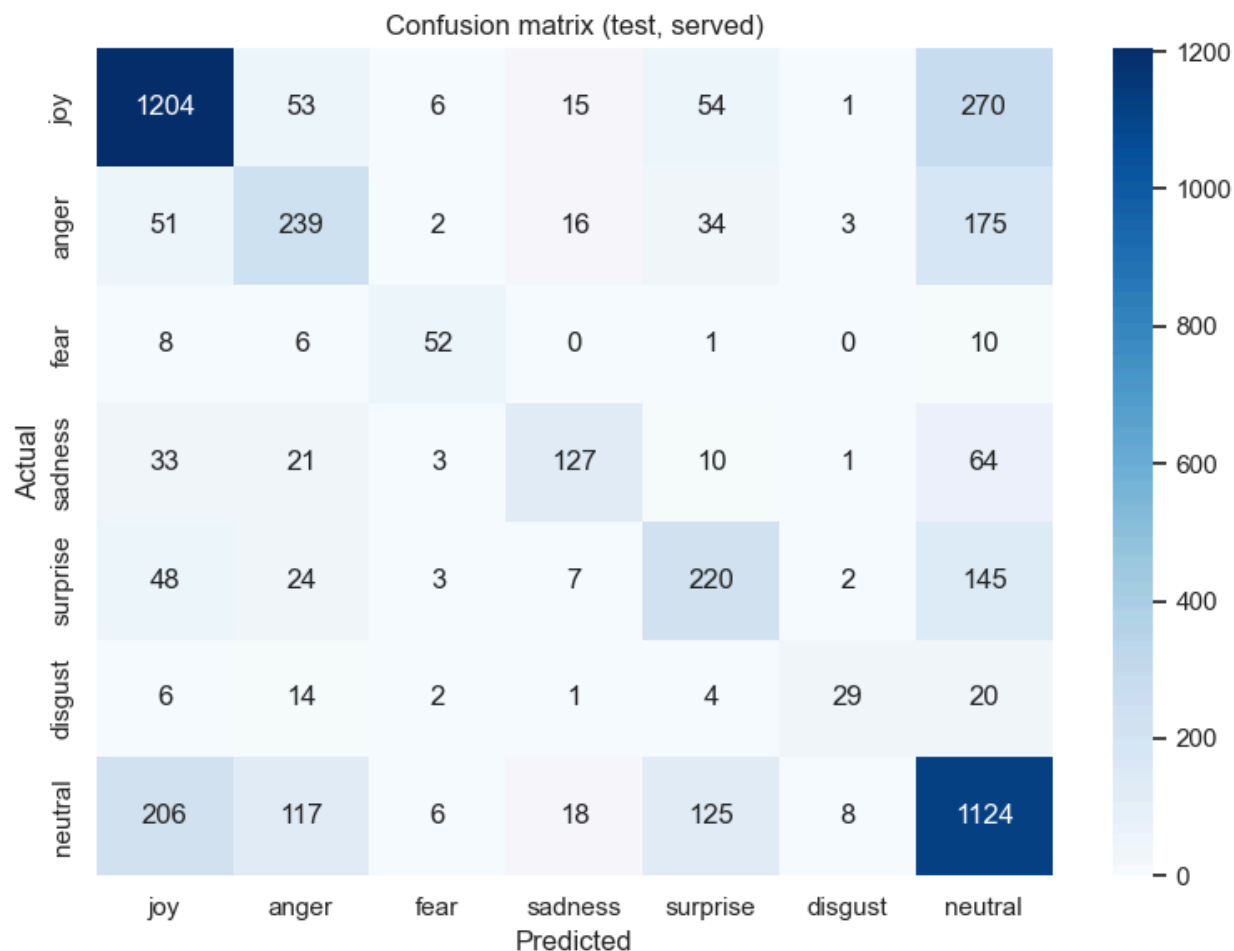


Figure 11: Confusion matrix on the held-out test set (served predictions). Joy/neutral dominate the diagonal; most remaining errors flow into neutral.

5.4 Crisis Safety Backoff

GoEmotions contains almost no self-harm language, so the TF-IDF model freely learned “suicide → joy” and the official splits cannot expose the failure (zero such rows). On the hand-labelled 38-row slice the results are unambiguous:

- **crisis rows (24):** model-only correct 0/24 → with backoff 24/24, every phrase triggered;
- **contrast rows (14):** unchanged by the backoff 14/14 and correct 14/14 ordinary texts (including “this job is killing me”) pass through untouched.

5.5 Automated Tests

python django_app/manage.py test runs **32 tests** (14 in accounts, 18 in predictor) covering registration/login/logout, form validation, prediction persistence, history scoping, artefact-missing errors, the seed command and the full crisis slice through the real serving path, all passing.

5.6 Key Observations

- Joy and neutral are the easiest classes to predict (large, homogeneous support); fear scores surprisingly well (0.6887) but on only 77 test rows, so the number is noisy.
- Anger, surprise and disgust remain the hardest classes (F1 0.48–0.49); they are frequently absorbed into neutral.
- Character n-grams matter: they capture misspellings and morphology that word tokens miss, and they were the largest single gain in the feature sweep.
- Calibration and decision boosts added +0.026 macro-F1 on test (0.5651 → 0.5911) at zero latency cost.
- Balanced class weights lifted minority-class recall substantially at a small cost to overall accuracy.

6. Challenges and Limitations

6.1 Challenges

Class imbalance: Joy and neutral together account for over 70% of training rows while fear and disgust each hold under 1.5%, making minority decision boundaries hard to learn.

Emotion subjectivity: The same text can convey different emotions depending on context and annotator interpretation, which injects label noise and places a natural ceiling on achievable accuracy.

Short text length: Comments average 12.6 words (median 12), leaving little context to separate closely related emotions such as anger/disgust or surprise/fear.

Multi-label ambiguity: ~16% of rows carry more than one emotion; they are dropped, reducing usable data and removing difficult edge cases.

Train–serve skew: Ensuring the preprocessing inside Django matched training exactly required sharing a single `clean_text` function and a single vectorizer artefact, plus tests

that pin the behaviour.

Evaluation hygiene: With only one dev and one test split, every hyperparameter (features, C, weights, calibration, boosts) had to be chosen on dev alone; the test set was scored exactly once at the end to keep the number honest.

Safety blind spot: The official splits contain no self-harm phrasing, so macro-F1 could never reveal that the model answered “suicide” with joy; a dedicated hand-labelled slice had to be built to surface and fix it.

6.2 Limitations

Objective narrowly missed: The final test macro-F1 is 0.5911, just below the 0.60 target (a gap of 0.009). Weak classes (anger 0.48, disgust 0.48, surprise 0.49) hold it back; disgust and fear have fewer than 80 test rows each, so their scores are noisy.

Domain shift: The model is trained only on Reddit comments and may not generalize to formal writing, reviews or other platforms.

No semantics: TF-IDF ignores word order and meaning; a transformer such as BERT (Devlin et al., 2019) would likely score higher but needs far more compute.

English only: No multilingual support.

Static model: It does not learn from new user inputs after deployment; retrains are offline and manual.

Single-label output: One dominant emotion per text; genuinely mixed texts get a ranked score list but only one verdict.

7. Conclusion

7.1 Summary

This project demonstrates the end-to-end development of an emotion detection system: from raw annotated data through EDA, preprocessing, model training, and rigorous evaluation to a deployable Django web application. Authenticated users submit free-form text and receive a fine-grained emotion classification with confidence scores across seven categories, going beyond the binary positive/negative sentiment analysis of most accessible tools. Every prediction is persisted per user, inspectable in the admin, and for high-stakes input protected by a rule-based safety layer, the tests hold in place.

7.2 Major achievements and learning outcomes

- Designed and validated a 28-to-7 emotion collapse mapping that balances class size with emotional granularity.
- Built a single NLP cleaning pipeline shared verbatim by training, evaluation, and serving, eliminating train–serve skew by construction.
- Trained a calibrated, boost-adjusted LinearSVC (110k word+char TF-IDF features) that scores 0.5911 test macro-F1, about 7.7× the 0.0767 baseline, with every hyperparameter chosen on dev only.
- Implemented a crisis-phrase backoff that turns a 0/24 correct safety result into 24/24 while leaving ordinary inputs bit-for-bit unchanged.
- Shipped a full Django application: auth, prediction UI with score bars, per-user history, admin, seed data and 32 automated tests.
- Gained practical experience with the complete ML lifecycle EDA, experiment design, error analysis, serialization, deployment and regression testing and with documenting it honestly (the missed 0.60 target included).

8. Future Scope

- Replace TF-IDF + LinearSVC with a fine-tuned DistilBERT or RoBERTa model to improve accuracy, especially on minority emotion classes.
- Expose the model as a REST API using FastAPI, letting the Django frontend consume predictions over HTTP and enabling third-party integrations.
- Integrate the Google Gemini API to generate plain-English explanations of why the model predicted a given emotion for a given input.
- Add multi-label prediction so texts expressing several emotions return a ranked list instead of a single verdict.
- Build a personal emotion-trend dashboard showing how a user's emotion distribution changes over time.
- Add batch prediction upload a CSV of texts, download the scored results.
- Deploy to a public cloud platform such as Railway or Render.
- Extend language support with multilingual pre-trained models.
- Expand the crisis phrase list from a curated tuple to a monitored, versioned configuration with its own regression slice.

References

- Demszky, D., Movshovitz-Attias, D., Ko, J., Cowen, A., Nemade, G. and Ravi, S. (2020) 'GoEmotions: a dataset of fine-grained emotions', in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4040–4054. Available at: <https://arxiv.org/abs/2005.00547> (Accessed: 8 September 2026).
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: pre-training of deep bidirectional transformers for language understanding', in *Proceedings of NAACL-HLT*, pp. 4171–4186.
- Django Software Foundation (2026) *Django documentation*. Available at: <https://docs.djangoproject.com/> (Accessed: 8 September 2026).
- Google Research (2021) *GoEmotions dataset* [GitHub]. Available at: <https://github.com/google-research/google-research/tree/master/goemotions> (Accessed: 8 September 2026).
- HuggingFace (2024) *google-research-datasets/go_emotions*. Available at: https://huggingface.co/datasets/google-research-datasets/go_emotions (Accessed: 8 September 2026).
- Salton, G. and Buckley, C. (1988) 'Term-weighting approaches in automatic text retrieval', *Information Processing & Management*, 24(5), pp. 513–523.

Appendices

How to Run

```
pip install -r requirements.txt

python django_app/manage.py migrate    # create the schema

python django_app/manage.py seed_db    # demo accounts + sample rows

python django_app/manage.py runserver  # http://127.0.0.1:8000/

python django_app/manage.py test accounts predictor  # 32 tests

python ml/train.py --refit              # retrain + rewrite the pickles

python ml/eval_crisis.py                # crisis slice report
```

Video link

 Project video.mp4